Algorithmique Fondamentale : Recherche Séquentielle et Dichotomique dans les Tableaux

Thème principal : Algorithmique et structures de données linéaires

Synthèse exécutive : Ce cours aborde les méthodes fondamentales pour retrouver une valeur au sein d'une structure de données linéaire (tableau). L'enjeu central repose sur la relation directe entre l'organisation des données (tableau trié ou non trié) et la performance temporelle de l'algorithme. La recherche séquentielle parcourt le tableau de manière exhaustive en complexité linéaire O(n) sans aucune précondition. À l'inverse, la recherche dichotomique s'appuie sur le paradigme 'diviser pour régner' pour réduire l'espace de recherche de moitié à chaque étape, garantissant une complexité logarithmique O(log n) à condition que le tableau soit préalablement trié. L'étudiant apprend à maîtriser l'exécution pas à pas de ces algorithmes, la manipulation des pointeurs d'indices et les critères d'arbitrage selon le contexte.

La Recherche Séquentielle (ou Linéaire)

La recherche séquentielle est la stratégie la plus intuitive et universelle pour localiser un élément dans un tableau. Elle consiste à passer en revue chaque case du tableau, du premier indice (0) jusqu'au dernier (n-1), en comparant à chaque étape la valeur contenue dans la case avec la valeur recherchée (la cible). L'algorithme s'arrête immédiatement en cas de succès (élément trouvé) ou lorsqu'il a parcouru la totalité du tableau sans succès. Si le tableau contient n éléments, le pire des cas se produit lorsque la valeur se trouve à la toute dernière position ou est totalement absente du tableau : l'algorithme doit alors effectuer exactement n comparaisons. La complexité est donc linéaire, notée O(n). Son principal avantage est qu'elle ne nécessite aucun pré-traitement ni organisation préalable des données.

- Parcours exhaustif élément par élément du début (0) vers la fin (n-1)
- Complexité temporelle linéaire O(n) au pire des cas
- Absence totale de précondition sur le tableau (trié ou non trié)

Cas concret enseignant : *Le professeur a utilisé l'analogie de la recherche d'une clé tombée dans un tas de feuilles mortes : n'ayant aucun ordre ni rangement dans le tas, il est obligatoire de soulever chaque feuille une à une, de la première à la dernière, jusqu'à trouver la clé.*

La Recherche Dichotomique (Principe du Diviser pour Régner)

La recherche dichotomique applique le principe algorithmique 'diviser pour régner'. Elle exige une condition préalable stricte : le tableau doit être trié au préalable. Au lieu d'examiner les éléments un par un, l'algorithme calcule l'indice du milieu de la zone de recherche courante : milieu = (debut + fin) // 2 (division entière). Il compare ensuite la valeur cible avec la valeur située à cet indice milieu. Trois cas se présentent : 1) Si T[milieu] == cible, l'élément est trouvé. 2) Si la cible est plus petite que T[milieu], la valeur ne peut se trouver que dans la moitié gauche : on restreint la recherche en déplaçant le pointeur fin à 'milieu - 1'. 3) Si la cible est plus grande, elle ne peut se trouver que dans la moitié droite : on déplace le pointeur debut à 'milieu + 1'. En éliminant la moitié des candidats restants à chaque étape, l'espace de recherche diminue de façon exponentielle, garantissant une complexité logarithmique O(log n). Par exemple, pour un tableau de 1 000 000 d'éléments triés, 20 comparaisons suffisent au maximum.

- Précondition absolue : le tableau doit être strictement trié au préalable
- Paradigme 'Diviser pour Régner' : élimination de 50 % de l'espace de recherche à chaque itération
- Gestion rigoureuse de trois pointeurs d'indices : debut, fin, et milieu = (debut + fin) // 2
- Complexité temporelle logarithmique O(log2 n) dans le pire des cas

Cas concret enseignant : *Le professeur a illustré cela avec le jeu du 'Devine un nombre entre 1 et 100' : proposer 1, 2, 3... est une stratégie séquentielle inefficace. Proposer 50 permet d'éliminer immédiatement la moitié des nombres si la réponse est 'c'est plus grand'. En réitérant cette stratégie, on trouve le nombre exact en 7 étapes maximum (car 2^7 = 128 > 100).*

Arbitrage entre Recherche Séquentielle et Dichotomique

En programmation et en algorithmique, choisir le bon algorithme dépend du contexte opérationnel. Bien que la recherche dichotomique soit immensément plus rapide que la recherche séquentielle pour de grands volumes, elle impose le tri préalable du tableau. Le meilleur algorithme de tri s'exécute en O(n log n). Ainsi, pour effectuer une unique recherche dans un tableau non trié, il est plus performant d'exécuter une recherche séquentielle O(n) que de trier d'abord le tableau en O(n log n) pour ensuite effectuer une recherche dichotomique en O(log n). En revanche, si le tableau est statique et doit subir des milliers de recherches, l'investissement initial du tri est largement amorti.

- Coût du tri préalable : Trier un tableau non trié nécessite O(n log n)
- Règle d'arbitrage : Si une seule recherche est effectuée sur un tableau non trié, la recherche séquentielle O(n) est plus rapide que Trier + Dichotomie
- Rentabilité des recherches multiples : Si l'on effectue de nombreuses recherches sur un tableau fixe, trier une seule fois le tableau est extrêmement rentable

Cas concret enseignant : *Analogie du dictionnaire : Organiser les mots par ordre alphabétique demande un travail considérable à l'imprimeur, mais ce tri permet à des millions de lecteurs de trouver un mot en quelques secondes pendant des années.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser la recherche dichotomique si elle est beaucoup plus rapide que la recherche séquentielle ?
Réponse du professeur : Parce que la recherche dichotomique exige que le tableau soit trié. Si le tableau n'est pas trié, le coût de son tri préalable annule le gain de vitesse, sauf si l'on effectue un très grand nombre de recherches répétées.

Question étudiant : Comment calcule-t-on l'indice du milieu si le sous-tableau contient un nombre pair d'éléments ?
Réponse du professeur : On utilise la division entière : milieu = (debut + fin) // 2. Le langage arrondit automatiquement à l'inférieur, ce qui ne pose aucun problème logique tant que la mise à jour des pointeurs exclut bien l'indice milieu testé (+1 ou -1).

■ Exercice & Démonstration d'Application

Énoncé : Soit le tableau trié T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] comportant n = 10 éléments (indices de 0 à 9). Appliquer pas à pas l'algorithme de recherche dichotomique pour localiser la valeur 42.

Résolution pas à pas :
Étape 0 (Initialisation) : debut = 0, fin = 9. Étape 1 : - Calcul du milieu : milieu = (0 + 9) // 2 = 4. - Valeur examinée : T[4] = 19. - Comparaison : 42 > 19. La cible est supérieure, on recherche dans la sous-partie droite. - Mise à jour du pointeur debut : debut = milieu + 1 = 5. Étape 2 : - Zone de recherche : debut = 5, fin = 9. - Calcul du milieu : milieu = (5 + 9) // 2 = 7. - Valeur examinée : T[7] = 42. - Comparaison : T[7] == 42. La valeur recherchée est trouvée !

Résultat : La valeur 42 a été localisée à l'indice 7 en seulement 2 comparaisons, alors qu'une recherche séquentielle aurait nécessité 8 comparaisons (du tableau[0]=3 jusqu'à tableau[7]=42).

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la condition impérative indispensable pour pouvoir appliquer une recherche dichotomique sur un tableau ?

R : Le tableau doit être strictement trié au préalable.

Q : Quelle est la complexité temporelle au pire des cas d'une recherche séquentielle sur un tableau de taille n ?

R : O(n), car on peut être amené à parcourir l'intégralité des n éléments.

Q : Combien de comparaisons au maximum la recherche dichotomique nécessite-t-elle pour localiser un élément dans un tableau trié de 1024 éléments ?

R : 10 comparaisons au maximum, car log2(1024) = 10.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765